Design Document

Student: B113040045許育菖

Library Functions

The important library functions in my project.

main()

`ftok()` : To create the key value for shared memory.

`shmget()`, `shmat()` : Used for creating and attaching shared memory segments.

`shmdt()`, `shmctl()` : Used for detaching and controlling shared memory segments.

`memset()` : Set the value and size of shared memory, I used it for formatting and clearing shared memory..

`strdup()`, `strtok()` : Handle the command input.

Spawn()

`snprintf()` : Write data into shared memory.

`fork()` : Create a child process.

Self Functions

In my project, I only change the “shell.c”. This file is modified from my shell in HW1, so there are still some interesting command, such as `rabbit`.

In “shell.c”, I created `ProcessFork()`, `Waitpid()`, `Exit()`, `Spawn()`, and `init_process_info()`. There is also a ***struct*** to save the information of each process.

ProcessFork()

```
// Fork a new process
pid_t ProcessFork(void)
{
    pid_t pid = fork();
    if (pid < 0)
```

```
    {
        return -1;  // Error forking
    }
    else if (pid == 0)
    {
        return 0;   // Child process returns 0
    }
    else
    {
        return pid;  // Parent process returns child PID
    }
}
```

Waitpid()

```
// Wait for a process to exit and return its exit status
int Waitpid(pid_t pid)
{
    if (pid < 0 || pid >= MAX_PROCS || process_info[pid].is_v
    {
        return -1;  // Invalid process ID or process does not
    }

    waitpid(pid, &(process_info[pid].exit_status), 0);


    return process_info[pid].exit_status;
}
```

Exit()

```
// Exit the current process with a given exit status
void Exit(int exit_status)
{
    pid_t pid = getpid();

    if (pid >= MAX_PROCS)
```

```
    {
        exit(EXIT_FAILURE);  // This should not happen
    }

    process_info[pid].exit_status = exit_status;
    process_info[pid].is_valid = 1;
}
```

Spawn()

```
// Spawn a new process with program name and argument
pid_t Spawn(const char *str, int arg) {
    pid_t pid;

    for(int i = 0; i < arg; i++)
    {
        pid = ProcessFork();

        if (pid < 0)
        {
            return -1;  // Failed to create new process
        }
        else if (pid == 0)
        {   /* Child process */
            printf("Reading from shared memory: %s\n", (char
        }
        else
        {   /* Parent process */
            // Write to shared memory
            snprintf(shared_dir, SHM_SIZE, "%s", str);
            printf("Message '%s' written to shared memory\n",

            if (!flag_back)
            {
                Exit(Waitpid(pid)); // Wait for child if runn
            }
            // Clear process info for reuse
            process_info[pid].exit_status = -1;
```

```
            process_info[pid].is_valid = 0;
            strcmp(shared_dir, NULL);
        }
    }

    return pid;
}
```

struct ProcessInfo

```
// Information for each process
struct ProcessInfo {
    int exit_status;
    int is_valid;
};

// Array to hold process information
struct ProcessInfo process_info[MAX_PROCS];
```

init_process_info()

```
// Initialize process information
void init_process_info()
{
    for (int i = 0; i < MAX_PROCS; i++)
    {
        process_info[i].exit_status = -1;
        process_info[i].is_valid = 0;
    }
}
```

Shared Memory

I use this way to create shared memory.

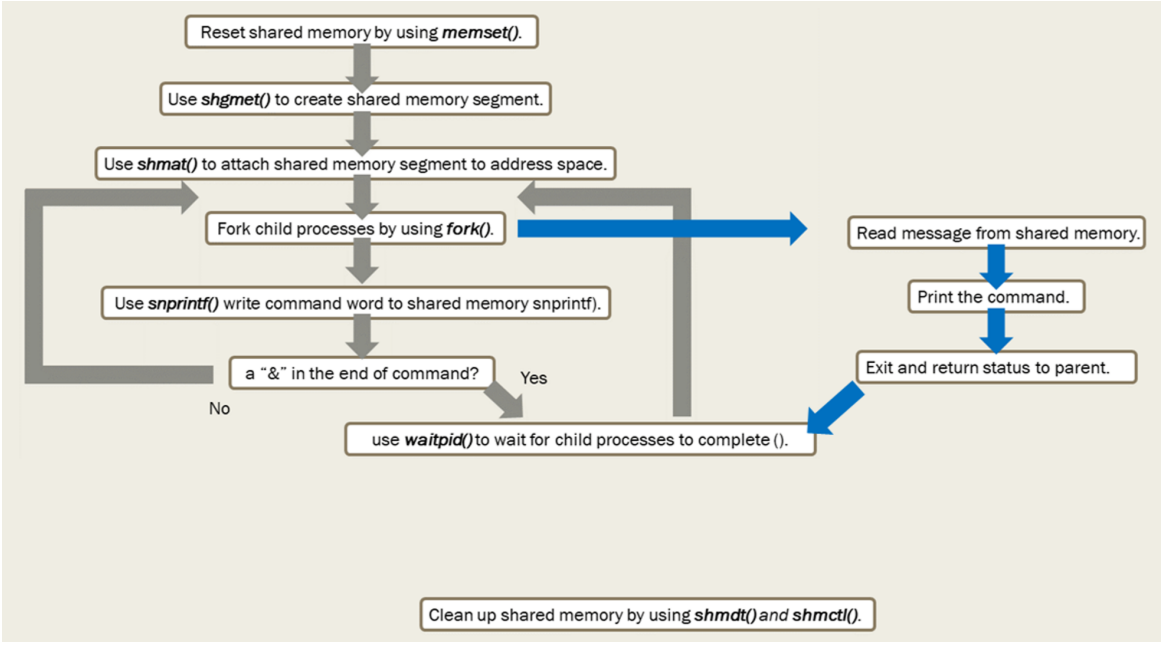
```
/* create shared memory */
```

```
int shmid;  //  store the return value from shmget()
key_t key;

key = ftok(".", 'a');
if(key == -1)
{
    perror("ftok() fause\n");
    exit(EXIT_FAILURE);
}

//  create shared memory
if ( (shmid = shmget (key, SHM_SIZE, 0666|IPC_CREAT)) == -1)
{
    perror("shmget: shmget failed");
    exit(EXIT_FAILURE);
}


// Map the shared memory segment to the address space of the
shared_dir = (char*)shmat(shmid, (void *)0, 0);
if(shared_dir == (char*)-1)
{
  perror("shared_dir fause\n");
  exit(EXIT_FAILURE);
}
```

While we type the command which is not the key word, such as “NSYSU”, “cd”, in the shell, the command word will be send to `Spawn()` .

Then it will create several child process, and parent process will write the command word into shared memory by `snprintf(shared_dir, SHM_SIZE, "%s", str);` . After that, child process will read the word from the memory and print to the screen by using pointer.

`printf("Reading from shared memory: %s\n", (char *)shared_dir);`



shared memory 流程圖

In my project, the mailbox is **undirectional**.

Since the parent process writes messages to shared memory, and the child process reads and processes them.

Q: How did you address memory mailbox synchronization? Describe in detail.

By using `waitpid()` in the parent process, we ensure that the parent waits for the child process to finish reading from shared memory before proceeding. This synchronization ensures that the parent process does not overwrite the shared memory while the child process is still reading from it.

Q: How did you prevent the same message being read by a single process multiple times? Describe in detail.

After the child process reads a message from shared memory, it exits immediately `exit(EXIT_SUCCESS)`. This prevents the child process from staying active and potentially reading the same message again. Additionally, the parent process cleans up the shared memory `memset()` after each use, ensuring that old data is cleared before writing new messages.

Traps

> Let’s see the traps used in my project.
>
> - `shmget()`: 子常式會傳回與指定 *Key* 參數相關聯的共用記憶體 ID。
> - `shmat()`: 將共用記憶體區段或對映檔附加至現行程序。
> - `shmdt()`: 分離共用記憶體區段。
> - `shmctl()`: 控制共用記憶體作業。
> - `fork()`: 建立child processprocess
> - `waitpid()`: 等待指頂的process執行完成，在此project的shell中，用於讓parent process等待child process執行完畢。

Challenges

1. It is pretty difficult to realize what I should to finish my project.
2. Through I know how the concept of the shared memory and how it work, but I don’t know how to make it be the truth in my program. There are too many system function and library that I have never seen before. Thus, I have to spend a lot of time surfing on the internet to understand how this function can do.